

IMPLEMENTASI SISTEM POS BERBASIS WEB UNTUK OPTIMALISASI EFISIENSI OPERASIONAL PADA USAHA LAUNDRY

LAPORAN PROYEK II

Diajukan untuk Memenuhi Kelulusan Matakuliah Proyek 2 pada Program Studi DIV
Teknik Informatika



DISUSUN OLEH :

Ismi Nabilah 714240056

Alifya Azzahra 714240011

**PROGRAM STUDI D IV TEKNIK INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS LOGISTIK DAN BISNIS INTERNASIONAL
BANDUNG**

2026

LEMBAR PENGESAHAN

IMPLEMENTASI SISTEM POS BERBASIS WEB UNTUK OPTIMALISASI EFISIENSI OPERASIONAL PADA USAHA LAUNDRY

Diajukan untuk Memenuhi Kelulusan Matakuliah Proyek 2
Program Studi D-IV Teknik Informatika

Disusun Oleh:

Alifya Azzahra 714240011

Ismi Nabilah 714240056

DOSEN PEMBIMBING

KOORDINATOR PROYEK 2

<u>Rolly Maulana Awangga, S.T., MT., CAIP., SFPC</u>	<u>Rd. Nuraini Siti Fathonah, S.S., M.Hum., SFPC</u>
NIK : 118.72.251	NIK : 117.86.219

MENYETUJUI,
KETUA PROGRAM STUDI DIV TEKNIK INFORMATIKA

Roni Andarsyah, S.T., M.Kom., SFPC
NIK. 115.88.193

SURAT PERNYATAAN TIDAK MELAKUKAN PLAGIARISME

Nama : Ismi Nabilah

NPM : 714240056

Program Studi : D IV Teknik Informatika

Judul : Implementasi Sistem POS Berbasis Web untuk Optimalisasi

Efisiensi Operasional pada Usaha Laundry

Menyatakan bahwa:

1. Proyek Pemrograman aplikasi (Proyek 2) saya ini adalah asli dan belum pernah diajukan untuk memenuhi kelulusan proyek 2 pada program studi D-IV Teknik informatika baik di Universitas Logistik Bisnis Internasional maupun di perguruan tinggi lainnya.
2. Proyek pemrograman aplikasi (Proyek 2) ini adalah murni gagasan, rumusan, dan peneliti saya sendiri tanpa bantuan orang lain, kecuali arahan pembimbing.
3. Dalam proyek pemrograman aplikasi (Proyek 2) ini tidak terdapat karya atau pendapat yang telah ditulis ataupun dipublikasi orang lain, kecuali secara tertulis dengan jelas dicantumkan sebagai acuan dalam naskah dengan disebutkan nama pengarang dan dicantumkan dalam daftar pustaka.
4. Pernyataan ini saya buat dengan sesungguhnya dan apabila di kemudian hari terdapat penyimpangan-penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik berupa pencabutan gelar yang telah diperoleh karena karya ini, serta sanksi lainnya sesuai dengan norma yang berlaku di perguruan tinggi lain.

Bandung, 2026

Yang Membuat Pernyataan,

Ismi Nabilah

NIM. 714240056

SURAT PERNYATAAN TIDAK MELAKUKAN PLAGIARISME

Nama : Alifya Azzahra

NPM : 714240011

Program Studi : D IV Teknik Informatika

Judul : Implementasi Sistem POS Berbasis Web untuk Optimalisasi

Efisiensi Operasional pada Usaha Laundry

Menyatakan bahwa:

1. Proyek Pemrograman aplikasi (Proyek 2) saya ini adalah asli dan belum pernah diajukan untuk memenuhi kelulusan proyek 2 pada program studi D-IV Teknik informatika baik di Universitas Logistik Bisnis Internasional maupun di perguruan tinggi lainnya.
2. Proyek Pemrograman aplikasi (Proyek 2) ini adalah murni gagasan, rumusan, dan peneliti saya sendiri tanpa bantuan orang lain, kecuali arahan pembimbing.
3. Dalam proyek Pemrograman aplikasi (Proyek 2) ini tidak terdapat karya atau pendapat yang telah ditulis ataupun dipublikasi orang lain, kecuali secara tertulis dengan jelas dicantumkan sebagai acuan dalam naskah dengan disebutkan nama pengarang dan dicantumkan dalam daftar pustaka.
4. Pernyataan ini saya buat dengan sesungguhnya dan apabila di kemudian hari terdapat penyimpangan-penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik berupa pencabutan gelar yang telah diperoleh karena karya ini, serta sanksi lainnya sesuai dengan norma yang berlaku di perguruan tinggi lain.

Bandung, 2026

Yang Membuat Pernyataan,

Alifya Azzahra

NIM. 714240011

ABSTRAK

Pencatatan transaksi pada operasional laundry yang masih bersifat manual seringkali menimbulkan risiko ketidakakuratan data dan kurangnya transparansi pembayaran. Penelitian ini bertujuan untuk mengembangkan Sistem *Point of Sale* (POS) WellClean Laundry berbasis web yang mengintegrasikan fitur manajemen pelanggan, layanan, dan transaksi otomatis. Sistem ini dibangun menggunakan bahasa pemrograman Go dengan arsitektur yang mendukung efisiensi pemrosesan data. Salah satu fitur unggulan yang diimplementasikan adalah integrasi pembayaran digital melalui QRIS yang didukung oleh mekanisme *webhook* untuk mendeteksi dana masuk secara *real-time* dan menutup jendela transaksi secara otomatis. Guna menjamin keandalan sistem, dilakukan audit pengujian unit (*unit testing*) menggunakan *tool go tool cover*. Hasil pengujian menunjukkan fungsionalitas krusial seperti *GopayHandler* dan *HashPassword* mencapai tingkat validitas sebesar 100.0%, dengan rata-rata keseluruhan *statements* sistem sebesar 15.3%. Implementasi ini terbukti dapat meningkatkan akurasi operasional dan memberikan kemudahan bagi kasir dalam memproses transaksi digital.

Kata Kunci: *Point of Sale, Go, QRIS, Unit Testing, Code Coverage.*

ABSTRAK

Manual transaction recording in laundry operations often leads to risks of data inaccuracy and a lack of payment transparency. This study aims to develop a web-based Point of Sale (POS) system for WellClean Laundry that integrates customer management, service management, and automated transaction features. The system is built using the Go programming language with an architecture designed for efficient data processing. One of the key features implemented is the integration of digital payments via QRIS, supported by a webhook mechanism to detect incoming funds in real-time and automatically close the transaction modal. To ensure system reliability, a unit testing audit was conducted using the go tool cover tool. The test results show that crucial functionalities such as GopayHandler and HashPassword achieved a 100.0

Keywords: *Point of Sale, Go, QRIS, Unit Testing, Code Coverage.*

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Allah SWT atas segala rahmat-Nya sehingga Laporan Proyek 2 yang berjudul "Implementasi Sistem POS Berbasis Web untuk Optimalisasi Efisiensi Operasional pada Usaha Laundry" dapat diselesaikan tepat pada waktunya. Laporan ini disusun untuk memenuhi salah satu syarat kelulusan mata kuliah Proyek 2 pada Program Studi D4 Teknik Informatika, Universitas Logistik dan Bisnis Internasional.

Isi laporan ini memaparkan perancangan, pengembangan, serta implementasi sistem POS yang digunakan untuk membantu operasional WellClean Laundry dalam mengelola data pelanggan, layanan, serta transaksi pembayaran secara digital. Harapannya, platform ini dapat membantu meningkatkan kualitas pelayanan, transparansi pembayaran melalui QRIS, serta mendukung proses pencatatan transaksi agar lebih tertata dan akurat.

Penulis menyadari bahwa laporan ini masih memiliki keterbatasan. Oleh karena itu, kritik dan saran yang membangun dari dosen pembimbing maupun pembaca sangat diharapkan demi penyempurnaan laporan ini. Semoga laporan ini dapat memberikan manfaat dan menjadi salah satu referensi dalam pengembangan sistem informasi di bidang layanan jasa laundry.

Bandung, Januari 2026

Penyusun,

Ismi Nabilah & Alifya Azzahra

DAFTAR ISI

DAFTAR ISI	vii
DAFTAR GAMBAR	ix
DAFTAR TABEL	x
BAB I PENDAHULUAN	I-1
1.1 Deskripsi Aplikasi	I-1
1.2 Identifikasi Masalah	I-1
1.3 Tujuan	I-2
1.4 Lingkup Dokumentasi	I-2
BAB II LANDASAN TEORI	II-1
2.1 Sistem Informasi	II-1
2.2 Poin of Sale (POS)	II-1
2.3 Usaha Laundry	II-1
2.4 Teori Usability dan User Experience	II-2
BAB III ANALISIS PERANCANGAN	III-1
3.1 Analisis	III-1
3.1.1 Analisis Fungsi Fungsional	III-1
3.1.2 Analisis Non Fungsional	III-1
3.2 Perancangan	III-2
3.2.1 Usecase Diagram	III-2
3.2.2 Antarmuka (UI)	III-2
3.2.3 Struktur Database	III-3
3.2.4 Algoritma fungsi	III-3
3.2.5 Flowchart	III-3
BAB IV IMPLEMENTASI DAN PENGUJIAN	IV-1
4.1 Pembahasan Hasil Implementasi	IV-1
4.1.1 Implementasi Antarmuka Pengguna (User Interface)	IV-1
4.1.2 Implementasi Logika Program (Bedah Kode)	IV-2
4.2 Pengujian dan Hasil Pengujian	IV-16
4.2.1 Pengujian Unit (Unit Testing)	IV-16
4.2.2 Hasil Pengujian Antarmuka dan Alur Fungsional	IV-16
BAB V KESIMPULAN DAN SARAN	V-1

5.1 Kesimpulan	V-1
5.2 Saran	V-1
DAFTAR PUSTAKA	V-1
LAMPIRAN	V-1
Bibliometrik	V-1

DAFTAR GAMBAR

1	Visualisasi Overview Bibliometrik (Biblioshiny)	V-1
---	---	-----

DAFTAR TABEL

BAB I

PENDAHULUAN

1.1 Deskripsi Aplikasi

Perkembangan teknologi informasi mendorong usaha kecil, termasuk laundry, untuk beralih dari pencatatan manual ke sistem digital. Salah satu solusi yang dikembangkan adalah Sistem Point of Sale (POS) Laundry berbasis web, yang berfungsi untuk mencatat transaksi, mengelola data pelanggan, memantau status cucian, serta menyusun laporan operasional secara terintegrasi. Dengan berbasis web, sistem ini dapat diakses secara fleksibel dari berbagai perangkat, sehingga meningkatkan efisiensi dan memudahkan pemilik usaha dalam melakukan monitoring [1].

Penelitian sebelumnya menegaskan bahwa aplikasi laundry berbasis web mampu menyederhanakan proses pencarian dan pengelolaan layanan laundry, sekaligus meningkatkan transparansi data dan akurasi pencatatan [2]. Selain itu, pendekatan berbasis pengguna dalam pengembangan sistem laundry terbukti dapat memperbaiki alur operasional dan meningkatkan kepuasan pelanggan, terutama melalui antarmuka yang mudah digunakan dan proses transaksi yang lebih cepat [3].

Studi lokal juga menunjukkan bahwa sistem informasi laundry berbasis web dapat membantu pemilik usaha dalam mengurangi kesalahan pencatatan, mempercepat proses transaksi, serta menghasilkan laporan yang lebih rapi dan terstruktur [4]. Penelitian lain menekankan bahwa penerapan sistem digital pada usaha kecil menengah (UMKM) berpengaruh positif terhadap efisiensi operasional dan kualitas layanan, sehingga mendukung keberlanjutan bisnis [5].

Berdasarkan kondisi tersebut, usaha laundry yang masih bergantung pada pencatatan manual berisiko menghadapi kendala seperti kesalahan hitung, keterlambatan informasi, dan pengelolaan data yang kurang optimal. Oleh karena itu, pengembangan Sistem POS Laundry Berbasis Website diharapkan menjadi solusi yang mampu meningkatkan efisiensi operasional, kualitas layanan, serta kepuasan pelanggan secara berkelanjutan.

1.2 Identifikasi Masalah

Berdasarkan latar belakang yang telah dijelaskan, maka permasalahan yang diidentifikasi dalam penelitian ini dapat dirumuskan sebagai berikut:

1. Bagaimana sistem POS Laundry berbasis web dapat menggantikan pencatatan manual yang masih digunakan?

2. Bagaimana sistem POS Laundry berbasis web dapat membantu pemilik dalam mengelola transaksi, pelanggan, dan laporan secara efisien?
3. Bagaimana antarmuka sistem POS Laundry berbasis web dapat dirancang agar mudah digunakan oleh karyawan maupun pemilik?
4. Bagaimana sistem POS Laundry berbasis web dapat meningkatkan akurasi pencatatan dan mengurangi kesalahan transaksi?

Rumusan masalah ini sejalan dengan penelitian terdahulu yang menekankan pentingnya digitalisasi layanan laundry untuk meningkatkan efisiensi, akurasi pencatatan, serta kepuasan pengguna [1][2][3][4][5].

1.3 Tujuan

Tujuan dari penelitian dan pengembangan sistem POS Laundry berbasis web ini adalah:

1. Merancang dan membangun sistem POS Laundry berbasis web untuk menggantikan pencatatan manual.
2. Menyediakan fitur transaksi, manajemen pelanggan, dan laporan yang terintegrasi.
3. Meningkatkan efisiensi operasional laundry melalui sistem digital yang cepat dan stabil.
4. Mengurangi kesalahan pencatatan serta meningkatkan akurasi data transaksi.

Tujuan ini didukung oleh penelitian sebelumnya yang menunjukkan bahwa sistem berbasis web dapat meningkatkan efisiensi operasional, memperbaiki kualitas layanan, dan mendukung keberlanjutan usaha laundry [1][3][4][5].

1.4 Lingkup Dokumentasi

Lingkup dokumentasi pada proyek ini dibatasi pada aspek aspek utama yang mendukung pengembangan Sistem POS Laundry berbasis web. Fokus utama meliputi:

1. Analisis kebutuhan sistem: mencakup identifikasi modul inti seperti transaksi, pelanggan, dan laporan.
2. Perancangan sistem berbasis web: meliputi desain antarmuka yang sederhana, responsif, dan mudah digunakan oleh karyawan maupun pelanggan.

3. Implementasi modul inti: transaksi laundry, manajemen pelanggan, serta laporan operasional dan keuangan.
4. Evaluasi awal sistem: dilakukan melalui uji coba terbatas untuk menilai efisiensi, akurasi pencatatan, dan kepuasan pengguna.

Batasan dokumentasi tidak mencakup pengembangan aplikasi mobile, integrasi notifikasi otomatis, maupun fitur tambahan di luar modul inti. Fokus diarahkan pada sistem berbasis web yang mendukung pencatatan transaksi, pemantauan status cucian, dan penyusunan laporan secara terstruktur. Penelitian sebelumnya menegaskan bahwa sistem laundry berbasis web dapat meningkatkan efisiensi operasional dan mengurangi kesalahan pencatatan [1][4][6]. Studi lain menunjukkan bahwa sistem berbasis web mampu menyediakan pemantauan status cucian secara digital, sehingga meningkatkan transparansi layanan [2][7]. Selain itu, pendekatan berbasis pengguna dalam pengembangan sistem laundry berkontribusi pada peningkatan kepuasan pelanggan melalui antarmuka yang jelas dan informatif [3][5].

BAB II

LANDASAN TEORI

2.1 Sistem Informasi

Sistem informasi adalah kombinasi dari teknologi, manusia, dan prosedur yang digunakan untuk mengumpulkan, memproses, menyimpan, serta mendistribusikan informasi guna mendukung pengambilan keputusan dan koordinasi dalam organisasi. Menurut Laudon dan Laudon dalam *Management Information Systems: Managing the Digital Firm* (2021), sistem informasi berperan penting dalam meningkatkan efisiensi operasional dan kualitas layanan melalui integrasi data dan proses bisnis. Hal ini sejalan dengan O'Brien dalam *Introduction to Information Systems* (2004) yang menekankan bahwa sistem informasi modern tidak hanya berfungsi sebagai alat pencatatan, tetapi juga sebagai sarana strategis untuk mendukung keberlangsungan bisnis di era digital. Dalam konteks modern, sistem informasi berbasis web menjadi pilihan utama karena dapat diakses secara fleksibel melalui berbagai perangkat, sehingga mendukung transparansi data dan integrasi proses bisnis [8][9].

2.2 Point of Sale (POS)

Point of Sale (POS) adalah sistem yang digunakan untuk mencatat transaksi penjualan secara digital dan terintegrasi. POS tidak hanya berfungsi sebagai alat pencatatan, tetapi juga sebagai sistem manajemen yang mampu menghubungkan data transaksi dengan laporan keuangan dan inventori. POS berbasis web memiliki keunggulan dibandingkan POS tradisional karena memungkinkan akses multi device, integrasi data secara real time, serta mempermudah pemilik usaha dalam melakukan monitoring. Penelitian menunjukkan bahwa sistem berbasis web dapat meningkatkan akurasi pencatatan dan efisiensi operasional [1][2].

2.3 Usaha Laundry

Usaha laundry sebagai bagian dari UMKM sering menghadapi kendala dalam pencatatan manual, seperti kesalahan transaksi, keterlambatan informasi, dan laporan yang tidak terstruktur. Digitalisasi layanan laundry melalui sistem POS berbasis web menjadi solusi untuk mengatasi permasalahan tersebut. Penelitian terdahulu menegaskan bahwa sistem laundry berbasis web mampu menyederhanakan proses pencatatan, mempercepat transaksi, serta meningkatkan kepuasan pelanggan melalui antarmuka yang mudah digunakan [3][4][5]. Studi lain juga menekankan bahwa digitalisasi laundry mendukung keberlanjutan usaha kecil dengan menyediakan laporan yang

lebih rapi dan terstruktur [6][7].

2.4 Teori Usability dan User Experience

Selain itu, teori usability menurut ISO 9241 11 menekankan bahwa sistem harus memenuhi aspek efektivitas, efisiensi, dan kepuasan pengguna. Prinsip usability sangat relevan dalam pengembangan POS Laundry berbasis web, karena antarmuka yang sederhana dan informatif akan mempermudah karyawan maupun pemilik dalam menggunakan sistem. Dengan demikian, pengembangan sistem POS Laundry berbasis web tidak hanya didukung oleh konsep sistem informasi dan POS, tetapi juga oleh teori usability yang memastikan sistem dapat digunakan secara optimal oleh penggunanya [3].

BAB III

ANALISIS PERANCANGAN

3.1 Analisis

3.1.1 Analisis Fungsi Fungsional

Analisis fungsional dilakukan untuk mengidentifikasi kebutuhan utama sistem POS Laundry berbasis web. Sistem ini dirancang agar dapat menggantikan pencatatan manual dengan menyediakan fitur inti sebagai berikut:

- Transaksi Laundry: mencatat pesanan pelanggan, jenis layanan (cuci, setrika, paket), jumlah, harga, dan status cucian.
- Manajemen Pelanggan: menyimpan data pelanggan, riwayat transaksi, serta mempermudah pencarian informasi.
- Laporan Operasional dan Keuangan: menghasilkan laporan harian, mingguan, dan bulanan terkait jumlah transaksi, pendapatan, serta status cucian.
- Pengelolaan Status Cucian: menampilkan status cucian (proses, selesai, diambil) secara real time.

Fungsi fungsi ini mendukung tujuan sistem untuk meningkatkan efisiensi, akurasi pencatatan, dan transparansi layanan.

3.1.2 Analisis Non Fungsional

Selain kebutuhan fungsional, sistem juga harus memenuhi kebutuhan non fungsional agar dapat digunakan secara optimal:

- Keamanan Data: sistem harus melindungi data pelanggan dan transaksi dari akses tidak sah.
- Kinerja Sistem: aplikasi harus mampu memproses transaksi dengan cepat dan stabil.
- Usability: antarmuka harus sederhana, responsif, dan mudah dipahami oleh karyawan maupun pemilik.
- Portabilitas: sistem berbasis web dapat diakses melalui berbagai perangkat (PC, laptop, smartphone).
- Reliabilitas: sistem harus mampu beroperasi secara konsisten tanpa gangguan yang signifikan.

3.2 Perancangan

Tahap perancangan merupakan proses penerjemahan hasil analisis kebutuhan ke dalam bentuk rancangan teknis yang akan digunakan sebagai acuan dalam pengembangan sistem POS Laundry berbasis web. Perancangan ini mencakup struktur menu, desain antarmuka pengguna, perancangan basis data, serta logika fungsi yang mendukung operasional laundry secara digital.

Tujuan utama dari perancangan adalah memastikan bahwa sistem yang dibangun sesuai dengan kebutuhan fungsional dan non fungsional yang telah diidentifikasi sebelumnya. Dengan rancangan yang terstruktur, pengembangan sistem dapat dilakukan secara efisien, terarah, dan minim risiko kesalahan implementasi. Setiap komponen yang dirancang baik dari sisi tampilan maupun proses internal harus mampu mendukung pencatatan transaksi, pengelolaan pelanggan, dan penyusunan laporan secara akurat dan real time.

3.2.1 Usecase Diagram

[gambar usecase]

3.2.2 Antarmuka (UI)

Desain antarmuka yang dirancang menggunakan prinsip responsif dan sederhana agar mudah digunakan oleh pengguna, baik melalui komputer maupun smartphone. Berdasarkan analisis kebutuhan, berikut adalah komponen utama dari antarmuka yang dirancang:

1. Halaman Login: Proses login melibatkan pengisian formulir terautentikasi untuk mendapatkan akses ke sistem yang memisahkan administrator dan pengguna.
2. Dashboard Admin: Area admin utama dengan menu navigasi untuk melihat data pengguna, data laundry, dan ringkasan laporan keuangan serta kinerja bisnis.
3. Dashboard Kasir: Ruang kerja utama untuk Kasir yang menyediakan fitur cepat untuk memasukkan transaksi baru, memeriksa status pelanggan, dan menganalisis data pelanggan.
4. Halaman Transaksi: Antarmuka untuk menentukan jenis layanan laundry (kiloan/satuan), menghitung total biaya secara otomatis, dan memproses pembayaran.
5. truk Transaksi: Tampilan bukti pembayaran yang dapat dicetak atau dikirim secara digital kepada pelanggan berisi detail layanan dan total biaya.

3.2.3 Struktur Database

[gambar]

Berdasarkan gambar 3.2.3 diatas, sistem ini menggunakan tiga tabel utama yang saling berelasi untuk mengelola data operasional laundry:

1. Tabel Pelanggan. Digunakan untuk menyimpan informasi identitas pelanggan seperti nama, alamat, telepon.
2. Tabel Layanan. Berisi daftar jenis jasa laundry yang ditawarkan, termasuk informasi seperti nama layanan, harga, satuan (misalnya per kg atau per potong), dan deskripsi.
3. Tabel Transaksi. Mencatat setiap pesanan yang dilakukan oleh pelanggan, termasuk total (biaya), status (status pengerjaan/pembayaran), dan berat (untuk laundry kiloan).

3.2.4 Algoritma fungsi

Dalam pengembangan sistem point-of-sale laundry berbasis web ini, algoritma fungsional digunakan untuk menentukan logika bisnis utama, seperti total biaya berdasarkan berat atau kuantitas, pemantauan status cucian secara real-time, dan laporan keuangan otomatis. Algoritma ini digunakan untuk memastikan proses kerja sistematis, akurat, dan meminimalkan kesulitan yang terkait dengan pekerjaan manual.

3.2.5 Flowchart

[gambar]

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Pembahasan Hasil Implementasi

Tahap implementasi merupakan fase di mana rancangan sistem yang telah dibuat sebelumnya direalisasikan ke dalam bentuk kode program menggunakan bahasa pemrograman Go (Golang) dan antarmuka berbasis web. Pada bagian ini, akan dijelaskan dua aspek utama implementasi, yaitu implementasi antarmuka pengguna (User Interface) untuk memberikan gambaran visual sistem, serta implementasi logika program (backend) yang menjelaskan alur kerja setiap fungsi secara mendetail.

4.1.1 Implementasi Antarmuka Pengguna (User Interface)

Implementasi antarmuka pada Sistem POS Laundry ini dirancang dengan prinsip usability untuk memastikan pengguna, baik admin maupun karyawan, dapat mengoperasikan sistem dengan mudah. Setiap halaman dirancang secara responsif guna mendukung fleksibilitas akses dari berbagai perangkat.

1. Halaman Otentikasi (Login)

[Gambar 4.1.1 1 Halaman Otentikasi (Login)] Halaman ini merupakan pintu masuk utama sekaligus garda depan keamanan sistem. Secara visual, halaman ini menyediakan form input untuk kredensial pengguna. Secara sistem, halaman ini berfungsi untuk memvalidasi identitas pengguna melalui pencocokan data yang ada pada database dengan teknik enkripsi tertentu guna menjaga kerahasiaan informasi.

2. Dashboard Operasional Utama

[Gambar 4.1.1 2 Halaman Dashboard] Dashboard ini merupakan pusat informasi dari seluruh aktivitas laundry. Implementasi halaman ini mencakup penyajian data transaksi harian, status pengerjaan cucian (Proses dan Selesai), total orderan, hingga total jumlah pendapatan. Informasi yang disajikan bersifat aktual (real-time) sehingga pemilik usaha dapat melakukan pemantauan operasional secara efisien tanpa harus melakukan pembaruan halaman secara manual.

3. Manajemen Pelanggan dan Transaksi

[Gambar 4.1.1 3 Halaman Manajemen Pelanggan]

[Gambar 4.1.1 4 Halaman Manajemen Transaksi]

Halaman ini mengimplementasikan fungsi CRUD (Create, Read, Update, Delete) untuk mengelola data pelanggan dan riwayat transaksi. Antarmuka pada bagian ini fokus pada akurasi input data untuk mengurangi risiko kesalahan pencatatan manual yang sering terjadi pada sistem konvensional.

4. Halaman Laporan Keuangan dan Riwayat Transaksi

[Gambar 4.1.1 5 Halaman Laporan]

Halaman ini berfungsi sebagai pusat dokumentasi finansial usaha laundry. Antarmuka laporan dirancang untuk memberikan fleksibilitas kepada pemilik dalam memantau laporan omset secara harian, bulanan, hingga tahunan. Dalam Visual dan Fungsi:

- Visual: Terdapat kartu ringkasan (summary cards) yang menampilkan total omset keseluruhan dan omset spesifik pada hari berjalan (seperti contoh pada Gambar 4.1.5 yang menunjukkan Omset Hari Ini sebesar Rp 1.250.000).
- Fungsi: Implementasi pada halaman ini memungkinkan pengguna untuk melihat tabel riwayat transaksi terakhir secara mendetail. Selain itu, terdapat tombol filtrasi laporan yang terhubung langsung dengan modul pengunduhan data (Export Excel).

5. Halaman Mode Kasir (Point of Sale)

[Gambar 4.1.1 6 Halaman Mode Kasir]

Mode Kasir (POS) merupakan fitur utama yang digunakan untuk melayani pelanggan secara langsung. Antarmuka ini dirancang dengan mengutamakan kecepatan proses input agar pelayanan tidak terhambat.

- Fungsi: Pada halaman ini, karyawan dapat memilih jenis layanan (laundry kiloan/satuan), memasukkan data pelanggan, dan menentukan metode pembayaran.
- Output: Setiap transaksi yang diselesaikan melalui Mode Kasir akan secara otomatis memperbarui saldo omset di halaman laporan dan memicu pengiriman data melalui sistem SSE (Server-Sent Events) ke dashboard utama.

4.1.2 Implementasi Logika Program (Bedah Kode)

Pada bagian ini akan dijelaskan alur logika dari fungsi-fungsi utama yang menjalankan Sistem POS Laundry. Penjelasan difokuskan pada bagaimana setiap baris

kode menangani data masuk (input), melakukan pengolahan (proses), hingga menghasilkan informasi yang dibutuhkan (output).

1. Modul Webhook Pembayaran (webhook.go) Berikut adalah potongan kode untuk fungsi GopayHandler dan StreamHandler:

[code ini ada di dalam kotak tabel dengan nama Table 4.1.2 1 Kode Webhook.go]

```
[package main
```

```
import ( "encoding/json" "fmt" "io" "net/http" "regexp" "strings" )
```

```
var pesanChan = make(chan string, 10)
```

```
func GopayHandler(w http.ResponseWriter, r *http.Request) bodyBytes, _ :=
io.ReadAll(r.Body)
```

```
var incoming struct NotificationText string 'json:"notification_text"' json.Unmarshal(bodyBytes
```

```
re := regexp.MustCompile('[\+]+') match := re.FindString(incoming.NotificationText)
```

```
finalText := incoming.NotificationText if match != "" angkaBersih := strings.ReplaceAll(match,
".", "") finalText = "Pembayaran Rp " + angkaBersih + " Berhasil!"
```

```
pesanChan <- finalText
```

```
fmt.Println("Berhasil kirim ke dashboard:", finalText) w.WriteHeader(http.StatusOK)
```

```
func StreamHandler(w http.ResponseWriter, r *http.Request) w.Header().Set("Content-
Type", "text/event-stream") w.Header().Set("Cache-Control", "no-cache") w.Header().Set("Con
"keep-alive") w.Header().Set("Access-Control-Allow-Origin", "*")
```

```
notify := r.Context().Done()
```

```
for select case <-notify: return case pesan := <-pesanChan: fmt.Fprintf(w,
"data:
```

```
if f, ok := w.(http.Flusher); ok f.Flush() ]
```

Penjelasan kode:

- (a) Input: Fungsi menerima objek `r *http.Request` yang berisi paket data JSON dengan kunci `notification_text` dari *payment gateway*.

- (a) Proses:

- i. Baris `io.ReadAll(r.Body)` membaca seluruh aliran data mentah yang masuk.

- ii. `json.Unmarshal` mengubah data mentah tersebut ke dalam struktur variabel agar teks notifikasi bisa diolah.
- iii. `regexp.MustCompile` dan `re.FindString` bekerja sama mencari pola angka di dalam teks untuk mendeteksi nominal pembayaran secara otomatis.
- iv. `strings.ReplaceAll` digunakan untuk membersihkan karakter titik agar nominal menjadi angka bulat yang siap disimpan.

(b) Output: Mengirimkan variabel `finalText` ke dalam `pesanChan` untuk diteruskan ke dashboard dan memberikan respon HTTP 200 OK ke server pengirim.

2. Modul Utama dan Manajemen Akun (`main.go`) Pada file `main.go`, seluruh logika bisnis dan integrasi database Supabase dikelola. Berikut adalah bedah baris kode berdasarkan fungsinya:

[kode dengan kotak tabel bernama Table 4.1.2 2 Kode Main.go]

```
[package main
```

```
import ( "bytes" "encoding/json" "fmt" "io" "net/http" "net/url" "strconv" "strings"
"time"
```

```
"golang.org/x/crypto/bcrypt"
```

```
"github.com/golang-jwt/jwt/v5" "github.com/xuri/excelize/v2" )
```

```
type User struct ID string `json:"id"` Username string `json:"username"` Pass-
word string `json:"password"` Role string `json:"role"`
```

```
type LoginRequest struct Username string `json:"username"` Password string
`json:"password"`
```

```
func enableCORS(w http.ResponseWriter) w.Header().Set("Access-Control-
Allow-Origin", "*") w.Header().Set("Access-Control-Allow-Headers", "Content-
Type, Authorization") w.Header().Set("Access-Control-Allow-Methods", "GET,
POST, PATCH, DELETE, OPTIONS")
```

```
func corsMiddleware(next http.HandlerFunc) http.HandlerFunc return func(w
http.ResponseWriter, r *http.Request) enableCORS(w) if r.Method == http.MethodOptions
w.WriteHeader(http.StatusOK) return next(w, r)
```

```
func validateToken(r *http.Request) (jwt.MapClaims, error) authHeader :=
r.Header.Get("Authorization") if authHeader == "" return nil, fmt.Errorf("missing
```

```

token")

tokenString := strings.TrimPrefix(authHeader, "Bearer ") token, err := jwt.Parse(tokenString,
func(token *jwt.Token) (interface, error) if ,ok := token.Method.(*jwt.SigningMethodHMAC)

if err != nil return nil, err

if claims, ok := token.Claims.(jwt.MapClaims); ok token.Valid return claims,
nil

return nil, fmt.Errorf("invalid token")

func loginHandler(w http.ResponseWriter, r *http.Request) if r.Method !=
http.MethodPost w.WriteHeader(http.StatusMethodNotAllowed) return

var req LoginRequest if err := json.NewDecoder(r.Body).Decode(req); err != nil
w.WriteHeader(http.StatusBadRequest) json.NewEncoder(w).Encode(map[string]string{"error":
"invalid body"}) return

// 1. Cari user di tabel public.users client := http.Client url := fmt.Sprintf("reqSup,
:= http.NewRequest("GET", url, nil) reqSup.Header.Set("apikey", APIKey) reqSup.Header.S
APIKey)

resSup, err := client.Do(reqSup) if err != nil w.WriteHeader(http.StatusInternalServerError)
json.NewEncoder(w).Encode(map[string]string{"error": "Supabase Connection
Error: " + err.Error()}) return defer resSup.Body.Close()

if resSup.StatusCode >= 400 var supError map[string]interface json.NewDecoder(resSup.Body
w.WriteHeader(resSup.StatusCode) json.NewEncoder(w).Encode(map[string]any{"error":
"Supabase Error", "details": supError}) return

var users []User if err := json.NewDecoder(resSup.Body).Decode(users); err !=
nil w.WriteHeader(http.StatusInternalServerError) json.NewEncoder(w).Encode(map[string]st
"Failed to decode users: " + err.Error()}) return

if len(users) == 0 w.WriteHeader(http.StatusUnauthorized) json.NewEncoder(w).Encode(map[
"User tidak ditemukan") return

user := users[0]

if !CheckPasswordHash(req.Password, user.Password) w.WriteHeader(http.StatusUnauthorized)
json.NewEncoder(w).Encode(map[string]string {"error": "Password salah", }) re-
turn

// 3. Buat JWT token := jwt.NewWithClaims(jwt.SigningMethodHS256, jwt.MapClaims
"sub": user.ID, "username": user.Username, "role": user.Role, "exp": time.Now().Add(time.Ho

```



```

* 24).Unix(), // 24 jam )

tokenString, err := token.SignedString([]byte(JWTSecret)) if err != nil w.WriteHeader(http.StatusUnauthorized)
return

w.Header().Set("Content-Type", "application/json") json.NewEncoder(w).Encode(map[string]string{
    "access_token": tokenString, "role": user.Role, })

func registerHandler(w http.ResponseWriter, r *http.Request)

claims, err := validateToken(r) if err != nil    fmt.Println("Gagal Verifikasi
Karena:", err)

w.WriteHeader(http.StatusUnauthorized) json.NewEncoder(w).Encode(map[string]string{"error":
err.Error()}) return

if claims["role"] != "admin" w.WriteHeader(http.StatusForbidden) json.NewEncoder(w).Encode(map[string]string{"error":
"Hanya admin yang bisa menambah kasir"}) return

var input struct { Username string `json:"username"` Password string `json:"password"`
Nama string `json:"nama"` }

if err := json.NewDecoder(r.Body).Decode(input); err != nil w.WriteHeader(http.StatusBadRequest)
return

hashed, err := HashPassword(input.Password) if err != nil w.WriteHeader(http.StatusInternalServerError)
return

userData := map[string]any {"username": input.Username, "password": hashed,
"nama": input.Nama, "role": "kasir", }

body, _ := json.Marshal(userData)

client := http.Client url := BaseURL + "/rest/v1/users" reqSup, _ := http.NewRequest("POST", url,
bytes.NewReader(body)) reqSup.Header.Set("Content-Type", "application/json")

resSup, err := client.Do(reqSup) if err != nil w.WriteHeader(http.StatusInternalServerError)
return defer resSup.Body.Close()

if resSup.StatusCode >= 400 w.WriteHeader(resSup.StatusCode) return

w.Header().Set("Content-Type", "application/json") w.WriteHeader(http.StatusCreated)
json.NewEncoder(w).Encode(map[string]string{"message": "Kasir berhasil didaftarkan"})

func getUsersHandler(w http.ResponseWriter, r *http.Request) claims, err := validateToken(r) if err != nil || claims["role"] != "admin" w.WriteHeader(http.StatusForbidden)

```

```

json.NewEncoder(w).Encode(map[string]string{"error": "Forbidden"}) return

client := http.Client // Coba ganti select=* dulu buat ngetes kalau kolom tertentu yang
bikin error url := BaseURL + "/rest/v1/users?select=*"

reqSup, _ := http.NewRequest("GET", url, nil) reqSup.Header.Set("apikey", APIKey) reqSup.Header.
Set("Authorization", "Bearer "+ APIKey)

resSup, err := client.Do(reqSup) if err != nil w.WriteHeader(http.StatusInternalServerError)
return defer resSup.Body.Close()

body, _ := io.ReadAll(resSup.Body)

// Kirim status code yang asli dari Supabase biar keliatan di browser w.Header().Set("Content-
Type", "application/json") w.WriteHeader(resSup.StatusCode) w.Write(body)

func deleteUserHandler(w http.ResponseWriter, r *http.Request) claims, err := vali-
dateToken(r) if err != nil || claims["role"] != "admin" w.WriteHeader(http.StatusUnauthorized)
return

id := r.URL.Query().Get("id") if id == "" w.WriteHeader(http.StatusBadRequest) re-
turn

client := http.Client reqSup, _ := http.NewRequest("DELETE", BaseURL + "/rest/v1/users?id =
eq." + id, nil) reqSup.Header.Set("apikey", APIKey) reqSup.Header.Set("Authorization", "Bearer "+
APIKey)

resSup, _ := client.Do(reqSup) w.WriteHeader(resSup.StatusCode)

func updateUserHandler(w http.ResponseWriter, r *http.Request) claims, err := vali-
dateToken(r) if err != nil || claims["role"] != "admin" w.WriteHeader(http.StatusUnauthorized)
return

id := r.URL.Query().Get("id") var input map[string]interface{} json.NewDecoder(r.Body).Decode(&input)

body, _ := json.Marshal(input) client := http.Client reqSup, _ := http.NewRequest("PATCH", BaseURL +
"/rest/v1/users?id = eq." + id, bytes.NewBuffer(body)) reqSup.Header.Set("apikey", APIKey) reqSup.
Header.Set("Content-Type", "application/json")

resSup, _ := client.Do(reqSup) w.WriteHeader(resSup.StatusCode)

func HashPassword(password string) (string, error) bytes, err := bcrypt.GenerateFromPassword([]byte(
password), bcrypt.DefaultCost) return string(bytes), err

func CheckPasswordHash(password, hash string) bool err := bcrypt.CompareHashAndPassword([]byte(
password), []byte(hash)) return err == nil

```

```

func dashboardHandler(w http.ResponseWriter, r *http.Request) claims, err := validateToken(r) if err != nil w.WriteHeader(http.StatusUnauthorized) json.NewEncoder(w).Encode(map[string]string{"error": err.Error()}) return

client := http.Client // Pakai Key Anon untuk ambil data transaksi transaksiURL := BaseURL + "/rest/v1/transaksi?select=*order=createdat.desc" reqTrans, := http.NewRequest("GET", transaksiURL, nil)

resTrans, err := client.Do(reqTrans) if err != nil w.WriteHeader(http.StatusInternalServerError) return defer resTrans.Body.Close()

var transaksiList []map[string]any if resTrans.StatusCode == http.StatusOK json.NewDecoder(resTrans.Body).Decode(&transaksiList) else transaksiList = []map[string]any

w.Header().Set("Content-Type", "application/json") json.NewEncoder(w).Encode(map[string]any{"ok": true, "user": claims["username"], "role": claims["role"], "transaksi": transaksiList, })

func pelangganHandler(w http.ResponseWriter, r *http.Request) ,err := validateToken(r) if err != nil w.WriteHeader(http.StatusUnauthorized) json.NewEncoder(w).Encode(map[string]string{"error": err.Error()}) return

query := r.URL.Query() searchTerm := query.Get("search")

bodyBytes, := io.ReadAll(r.Body) r.Body = io.NopCloser(bytes.NewBuffer(bodyBytes))

filterQuery := "" if searchTerm != "" filterQuery = fmt.Sprintf("or=(nama.ilike.*%s)", searchTerm)

fullURL := BaseURL + "/rest/v1/pelanggan?select=*" if r.Method == http.MethodGet fullURL += filterQuery

page, := strconv.Atoi(query.Get("page")) limit, := strconv.Atoi(query.Get("limit"))

query.Del("page") query.Del("limit") query.Del("search")

if len(query) > 0 fullURL += "?" + query.Encode()

reqSup, := http.NewRequest(r.Method, fullURL, bytes.NewBuffer(bodyBytes))

reqSup.Header.Set("Content-Type", "application/json") reqSup.Header.Set("apikey", APIKey) reqSup.Header.Set("Authorization", "Bearer "+APIKey)

if r.Method == http.MethodGet page > 0 if limit < 1 limit = 10 from := (page - 1) * limit to := from + limit - 1 reqSup.Header.Set("Range", fmt.Sprintf("bytes=%d-%d", from, to))

client := http.Client resSup, err := client.Do(reqSup) if err != nil w.WriteHeader(http.StatusInternalServerError) return defer resSup.Body.Close()

```

```

resBody, _ := io.ReadAll(resSup.Body)w.Header().Set("Content-Type", "application/json")w.Wr

func transaksiHandler(w http.ResponseWriter, r *http.Request) {err := validateToken(r)if err !=
nilw.WriteHeader(http.StatusUnauthorized)json.NewEncoder(w).Encode(map[string]string{"erro

query := r.URL.Query() page, _ := strconv.Atoi(query.Get("page"))limit, _ := strconv.Atoi(query.Get("
query.Get("search"))

bodyBytes, _ := io.ReadAll(r.Body)r.Body = io.NopCloser(bytes.NewBuffer(bodyBytes))

fullURL := BaseURL + "/rest/v1/transaksi" params := url.Values params.Add("select",
"*)" params.Add("order", "created_at.desc")

if r.Method == http.MethodGet searchTerm != "" params.Add("kode", "ilike.*"+searchTerm+"*")

query.Del("page") query.Del("limit") query.Del("search") query.Del("order") for k, v
:= range query params.Add(k, v[0]) fullURL += "?" + params.Encode()

client := http.Client reqSup, _ := http.NewRequest(r.Method, fullURL, bytes.NewBuffer(bodyBytes)
Type", "application/json")reqSup.Header.Set("apikey", APIKey)reqSup.Header.Set("Authorization
APIKey)

if r.Method == http.MethodGet page > 0 if limit < 1 limit = 10 from := (page - 1) *
limit to := from + limit - 1 reqSup.Header.Set("Range", fmt.Sprintf("

resSup, err := client.Do(reqSup) if err != nil w.WriteHeader(http.StatusInternalServerError)
return defer resSup.Body.Close()

resBody, _ := io.ReadAll(resSup.Body)w.Header().Set("Content-Type", "application/json")w.Wr

func layananHandler(w http.ResponseWriter, r *http.Request) {claims, err := validate-
Token(r) if err != nil w.WriteHeader(http.StatusUnauthorized)json.NewEncoder(w).Encode(map[stri
err.Error()) return

method := r.Method if method == http.MethodPost || method == http.MethodDelete ||
method == http.MethodPatch || method == http.MethodPut role, _ := claims["role"].(string)if role !=
"admin"w.WriteHeader(http.StatusForbidden)json.NewEncoder(w).Encode(map[string]string{"e

query := r.URL.Query() page, _ := strconv.Atoi(query.Get("page"))limit, _ := strconv.Atoi(query.Get("
query.Get("search"))

bodyBytes, _ := io.ReadAll(r.Body)r.Body = io.NopCloser(bytes.NewBuffer(bodyBytes))

fullURL := BaseURL + "/rest/v1/layanan" params := url.Values params.Add("select",
"*)")

```

```

if method == http.MethodGet searchTerm != "" params.Add("nama", "ilike.*"+searchTerm+"*")

query.Del("page") query.Del("limit") query.Del("search") for k, v := range query
params.Add(k, v[0])

fullURL += "?" + params.Encode()

client := http.Client reqSup, := http.NewRequest(method, fullURL, bytes.NewBuffer(bodyBytes))
Type", "application/json") reqSup.Header.Set("apikey", APIKey) reqSup.Header.Set("Authorization",
APIKey)

if method == http.MethodGet page > 0 if limit < 1 limit = 10 from := (page - 1) *
limit to := from + limit - 1 reqSup.Header.Set("Range", fmt.Sprintf("

resSup, err := client.Do(reqSup) if err != nil w.WriteHeader(http.StatusInternalServerError)
return defer resSup.Body.Close()

resBody, := io.ReadAll(resSup.Body) w.Header().Set("Content-Type", "application/json") w.Wr

type Transaksi struct Kode string `json:"kode"` CreatedAt time.Time `json:"created_at"` SelesaiAt *
time.Time `json:"selesai_at"` PelangganNama string `json:"pelanggan_nama"` LayananNama string `j
"layanan_nama"` Berat float64 `json:"berat"` HargaSatuan float64 `json:"harga_satuan"` Total float
"total"` MetodePembayaran string `json:"metode_pembayaran"` Status string `json:
"status"`

func laporanHandler(w http.ResponseWriter, r *http.Request) {err := validateToken(r) if err !=
nil w.WriteHeader(http.StatusUnauthorized) return

tipe := r.URL.Query().Get("type") client := http.Client

if tipe == "riwayat" transaksiURL := BaseURL + "/rest/v1/transaksi_wib" + "?status =
eq.selesai" + "select = kode, created_at, selesai_at, berat, total, metode_pembayaran, status, pelanggan
"order = selesai_at.desc"

req, := http.NewRequest("GET", transaksiURL, nil) req.Header.Set("apikey", APIKey) req.Header.
APIKey)

res, err := client.Do(req) if err != nil w.WriteHeader(http.StatusInternalServerError)
return defer res.Body.Close()

var raw []map[string]any json.NewDecoder(res.Body).Decode(raw)

trx := []Transaksi for , item := range raw {t := Transaksi

```

```
t.Kode = fmt.Sprintf(item["kode"]) t.Status = fmt.Sprintf(item["status"]) t.MetodePembayaran
= fmt.Sprintf(item["metode_pembayaran"])
```

```
if p, ok := item["pelanggan"].(map[string]any); ok t.PelangganNama = fmt.Sprintf(p["nama"])
```

```
if l, ok := item["layanan"].(map[string]any); ok t.LayananNama = fmt.Sprintf(l["nama"])
```

```
if s, ok := item["selesai_at"].(string); oks! = "" if t, err := time.Parse(time.RFC3339, s); err == nil.
```

```
if total, ok := item["total"].(float64); ok t.Total = total
```

```
trx = append(trx, t)
```

```
w.Header().Set("Content-Type", "application/json") json.NewEncoder(w).Encode(map[string]any{"data":
trx}) return
```

```
if tipe == "keuangan" loc, _ := time.LoadLocation("Asia/Jakarta") today := time.Now().In(loc).Format("2006-01-02")
```

```
getSum := func(url string) float64 req, _ := http.NewRequest("GET", url, nil) req.Header.Set("apikey",
APIKey)
```

```
res, err := client.Do(req) if err != nil return 0 defer res.Body.Close()
```

```
var list []map[string]any json.NewDecoder(res.Body).Decode(list)
```

```
var sum float64 for i, v := range list if t, ok := v["total"].(float64); ok sum += t return sum
```

```
// TOTAL OMSET (SEMUA) totalURL := BaseURL + "/rest/v1/transaksi_wib" +
"?status = eq.seleksi = total"
```

```
// OMSET HARIAN (WIB FIX) dailyURL := BaseURL + "/rest/v1/transaksi_wib" +
"?status = eq.selesai" + "selesai_angkal_wib = eq." + today + "select = total"
```

```
w.Header().Set("Content-Type", "application/json") json.NewEncoder(w).Encode(map[string]any{"total_omset": getSum(totalURL), "omset_harian": getSum(dailyURL),}) return
```

```
w.WriteHeader(http.StatusBadRequest)
```

```
func laporanExportHandler(w http.ResponseWriter, r *http.Request) claims, err :=
validateToken(r) if err != nil w.WriteHeader(http.StatusUnauthorized) return
```

```
role, _ := claims["role"].(string) if role != "admin" w.WriteHeader(http.StatusForbidden) return
```

```
tipe := r.URL.Query().Get("type") periode := r.URL.Query().Get("periode")
```

```
loc, _ := time.LoadLocation("Asia/Jakarta") var start, end time.Time
```

```

switch periode case "harian": valDate := r.URL.Query().Get("date") tgl, := time.ParseInLocation("2006-01-02", valDate, loc) start = time.Date(tgl.Year(), tgl.Month(), tgl.Day(), 0, 0, 0, 0, loc) end = time.Date(tgl.Year(), tgl.Month(), tgl.Day(), 23, 59, 59, 0, loc)

case "bulanan": y, := strconv.Atoi(r.URL.Query().Get("year")) m, := strconv.Atoi(r.URL.Query().Get("month")) start = time.Date(y, time.Month(m), 1, 0, 0, 0, 0, loc) end = start.AddDate(0, 1, -1) end = time.Date(end.Year(), end.Month(), end.Day(), 23, 59, 59, 0, loc)

case "tahunan": y, := strconv.Atoi(r.URL.Query().Get("year")) start = time.Date(y, 1, 1, 0, 0, 0, 0, loc) end = time.Date(y, 12, 31, 23, 59, 59, 0, loc)

default: now := time.Now().In(loc) start = time.Date(now.Year(), now.Month(), now.Day(), 0, 0, 0, 0, loc) end = start.AddDate(0, 0, 1)

filter := fmt.Sprintf("selesai_anggal_wib = gte.start.Format(\"2006-01-02\"), end.Format(\"2006-01-02\"))

var transaksiURL string if tipe == "riwayat" transaksiURL = BaseURL + "/rest/v1/transaksi_wib?status=" + tipe + "&seleksi=" + kode + "&selesai_anggal_wib=" + selesai_anggal_wib + "&berat=" + berat + "&total=" + total + "&metode_pembayaran=" + metode_pembayaran + "&status=" + status + "&pelanggan=" + pelanggan + "&nama=" + nama

transaksiURL += "" + filter

client := http.Client req, := http.NewRequest("GET", transaksiURL, nil) req.Header.Set("apikey", APIKey)

res, err := client.Do(req) if err != nil w.WriteHeader(http.StatusInternalServerError)
return defer res.Body.Close()

var raw []map[string]any json.NewDecoder(res.Body).Decode(raw)

var trx []Transaksi var totalOmsetPeriode float64

for , item := rangeraw t := Transaksi Kode = item["kode"] t.Status = item["status"]

if p, ok := item["pelanggan"]; ok t.PelangganNama = item["nama"]
if l, ok := item["layanan"]; ok t.LayananNama = item["nama"]

if s, ok := item["selesai_anggal_wib"]; ok oks! = "" if tm, err := time.Parse(time.RFC3339, s); err == nil t.SelesaiAnggalWib = tm

if total, ok := item["total"]; ok t.Total = total else if totalStr, ok := item["total"]; ok t.Total, _ = strconv.ParseFloat(totalStr, 64)

if berat, ok := item["berat"]; ok t.Berat = berat if t.Berat > 0 t.HargaSatuan = t.Total / t.Berat

totalOmsetPeriode += t.Total trx = append(trx, t)

```

```

f := excelize.NewFile() sheet := "Laporan" f.SetSheetName("Sheet1", sheet)

headerStyle, _ := f.NewStyle(excelize.StyleFont : excelize.FontBold : true, Color : "FFFFFF", Fill
f.NewStyle(excelize.StyleFont : excelize.FontBold : true, Fill : excelize.FillType : "pattern", Color

if tipe == "riwayat" headers := []string{"Kode", "Tanggal Selesai", "Pelanggan",
"Layanan", "Berat", "Harga Satuan", "Total", "Metode", "Status" for i, h := range head-
ers col := string(rune('A' + i)) f.SetCellValue(sheet, col+"1", h) f.SetCellStyle(sheet,
col+"1", col+"1", headerStyle) f.SetColWidth(sheet, col, col, 18)

for i, t := range trx row := strconv.Itoa(i + 2) f.SetCellValue(sheet, "A"+row, t.Kode)
if t.SelesaiAt != nil f.SetCellValue(sheet, "B"+row, t.SelesaiAt.In(loc).Format("02-
01-2006")) f.SetCellValue(sheet, "C"+row, t.PelangganNama) f.SetCellValue(sheet,
"D"+row, t.LayananNama) f.SetCellValue(sheet, "E"+row, t.Berat) f.SetCellValue(sheet,
"F"+row, t.HargaSatuan) f.SetCellValue(sheet, "G"+row, t.Total) f.SetCellValue(sheet,
"H"+row, t.MetodePembayaran) f.SetCellValue(sheet, "I"+row, t.Status)

lastRow := strconv.Itoa(len(trx) + 2) f.SetCellValue(sheet, "F"+lastRow, "TOTAL")
f.SetCellValue(sheet, "G"+lastRow, totalOmsetPeriode) f.SetCellStyle(sheet, "F"+lastRow,
"G"+lastRow, totalStyle)

else f.SetCellValue(sheet, "A1", "Tanggal Selesai") f.SetCellValue(sheet, "B1", "Om-
set (Rp)") f.SetCellStyle(sheet, "A1", "B1", headerStyle) f.SetColWidth(sheet, "A",
"B", 25)

for i, t := range trx row := strconv.Itoa(i + 2) if t.SelesaiAt != nil f.SetCellValue(sheet,
"A"+row, t.SelesaiAt.In(loc).Format("02-01-2006")) f.SetCellValue(sheet, "B"+row,
t.Total)

lastRow := strconv.Itoa(len(trx) + 2) f.SetCellValue(sheet, "A"+lastRow, "TOTAL
OMSET PERIODE INI") f.SetCellValue(sheet, "B"+lastRow, totalOmsetPeriode)
f.SetCellStyle(sheet, "A"+lastRow, "B"+lastRow, totalStyle)

var buf bytes.Buffer f.Write(buf)

w.Header().Set("Content-Type", "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet")
w.Header().Set("Content-Disposition", fmt.Sprintf("attachment; filename=Laporan_w. Write(buf.Bytes())

func main() http.HandleFunc("/login", corsMiddleware(loginHandler)) http.HandleFunc("/register",
corsMiddleware(registerHandler)) http.HandleFunc("/users", corsMiddleware(getUsersHandler))
http.HandleFunc("/users/delete", corsMiddleware(deleteUserHandler)) http.HandleFunc("/users/update",
corsMiddleware(updateUserHandler)) http.HandleFunc("/dashboard", corsMiddle-
ware(dashboardHandler)) http.HandleFunc("/pelanggan", corsMiddleware(pelangganHandler))

```



```

http.HandleFunc("/transaksi", corsMiddleware(transaksiHandler)) http.HandleFunc("/layanan",
corsMiddleware(layananHandler)) http.HandleFunc("/laporan", corsMiddleware(laporanHandler))
http.HandleFunc("/laporan/export", corsMiddleware(laporanExportHandler)) http.HandleFunc("/api/
gopay", GopayHandler) http.HandleFunc("/api/stream", StreamHandler)

fmt.Println("Server WellClean Laundry jalan di: http://localhost:8080") http.ListenAndServe(":8080",
nil) ]

```

A. Fungsi Otentikasi (loginHandler) Fungsi ini menangani proses verifikasi identitas pengguna untuk masuk ke sistem.

- Input: Menerima objek LoginRequest yang berisi username dan password dari body permintaan klien.
- Proses:
 - Baris `json.NewDecoder(r.Body).Decode(req)` membaca input dari pengguna.
 - Program melakukan query ke Supabase (`/rest/v1/users?username=eq.`) untuk mencari data pengguna.
 - Jika user ditemukan, baris `CheckPasswordHash` akan membandingkan password input dengan hash di database.
 - Baris `jwt.NewWithClaims` akan membungkus informasi user (ID, Role) ke dalam token keamanan yang berlaku selama 24 jam.
- Output: Mengembalikan `access_token` dalam format JSON jika login berhasil.

B. Fungsi Registrasi Kasir (registerHandler) Fungsi ini memastikan hanya Admin yang bisa menambahkan data karyawan baru.

- Input: Data JSON berupa username, password, dan nama lengkap kasir.
- Proses:
 - Baris `validateToken(r)` melakukan validasi apakah pengirim data adalah Admin yang sah.
 - Baris `HashPassword(input.Password)` menjalankan enkripsi Bcrypt terhadap password kasir baru.

- Baris `http.NewRequest("POST", ...)` mengirimkan data yang sudah terenkripsi ke tabel `users` di Supabase.
 - Output: Pesan sukses status 201 Created dan penyimpanan data kasir ke database.
- C. Fungsi Manajemen Pelanggan (`pelangganHandler`) Mengelola data pelanggan yang mencakup fitur pencarian (`searching`) dan pembatasan data (`pagination`).
- Input: Parameter URL `search` untuk kata kunci nama/telepon, serta `page` dan `limit`.
 - Proses:
 - Logika `fmt.Sprintf("or=(nama.ilike.*`
 - Baris `reqSup.Header.Set("Range", ...)` mengatur batasan data yang diambil dari server agar aplikasi tetap ringan (`pagination`).
 - Output: Daftar pelanggan dalam format JSON yang sesuai dengan kriteria pencarian.
- D. Fungsi Ekspor Laporan Excel (`laporanExportHandler`) Fungsi ini mengubah data transaksi menjadi dokumen fisik yang rapi.
- Input: Parameter `type` (riwayat/keuangan) dan periode (harian/bulanan/tahunan).
 - Proses:
 - Baris `excelize.NewFile()` membuat instance dokumen Excel baru.
 - Logika `f.SetCellStyle` memberikan format visual pada dokumen seperti warna header ungu (4B0082) dan teks tebal.
 - Sistem melakukan iterasi (`looping`) pada data transaksi dan menuliskan nilai Kode, Berat, dan Total ke dalam sel Excel menggunakan `f.SetCellValue`.
 - Output: File biner berformat `.xlsx` yang dikirim ke browser dengan header `Content-Disposition: attachment`.
- E. Fungsi Inisialisasi Server (`main`) Merupakan titik masuk utama (`entry point`) aplikasi.
- Input: Pendaftaran jalur URL (`routing`) sistem.

- Proses: Baris `http.HandleFunc` menghubungkan setiap alamat URL (seperti `/login`, `/transaksi`, `/laporan`) ke fungsi penanganan (handler) masing-masing, serta membungkusnya dengan `corsMiddleware` agar bisa diakses dari berbagai domain.
- Output: Server aktif dan berjalan pada alamat `http://localhost:8080`.

4.2 Pengujian dan Hasil Pengujian

Tahap pengujian dilakukan untuk memvalidasi apakah setiap fungsi pada Sistem POS Laundry telah berjalan sesuai dengan rancangan yang ditetapkan. Pengujian ini mencakup validasi terhadap masukan (input), alur kerja (proses), dan hasil yang dikeluarkan oleh aplikasi (output).

4.2.1 Pengujian Unit (Unit Testing)

Pengujian unit dilakukan untuk memvalidasi setiap fungsi secara spesifik. Berikut adalah hasil pengujian menggunakan perintah `go tool cover` yang menunjukkan persentase baris kode yang berhasil dijalankan dalam skenario uji:

[Gambar 4.2.1 1 Hasil Tes Codecoverage] Analisis Hasil Pengujian: Berdasarkan data audit pada gambar di atas, dapat disimpulkan beberapa hal sebagai berikut:

- Fungsi Keamanan: Fungsi `enableCORS` dan `CheckPasswordHash` mencapai cakupan 100.0
- Logika Pembayaran: Fungsi `GopayHandler` pada modul `webhook` mencapai 100.0
- Logika Real-time: Fungsi `StreamHandler` mencapai 72.7
- Efektivitas Sistem: Secara keseluruhan, sistem mencapai total cakupan 15.3

4.2.2 Hasil Pengujian Antarmuka dan Alur Fungsional

Pengujian ini dilakukan secara manual untuk memastikan setiap elemen antarmuka (user interface) dan alur bisnis pada Sistem POS WellClean Laundry berjalan sesuai dengan rancangan.

1. Pengujian Otentikasi (Login)

[Gambar 4.2.1 2 Otentikasi Login]

Proses pengujian dimulai pada gerbang awal keamanan sistem, yaitu halaman login. Pada tahap ini, admin memasukkan kredensial berupa username dan pass-

word yang kemudian diproses oleh sistem melalui fungsi loginHandler yang memiliki cakupan pengujian sebesar 13.0

2. Pengujian Monitoring Dashboard

[Gambar 4.2.1 3 Monitoring Dashboard]

Setelah berhasil melakukan proses login, sistem secara otomatis mengarahkan pengguna ke halaman dashboard utama yang berfungsi sebagai pusat informasi operasional. Halaman ini dikelola oleh fungsi dashboardHandler dengan persentase cakupan kode sebesar 23.8

3. Pengujian Manajemen Data (Pelanggan Layanan)

[Gambar 4.2.1 4 Manajemen Data Pelanggan]

[Gambar 4.2.1 5 Manajemen Data Layanan]

Pengujian selanjutnya berfokus pada kemampuan sistem dalam mengelola data master, yaitu informasi pelanggan dan kategori layanan laundry. Dengan memanfaatkan fungsi pelangganHandler yang memiliki coverage 11.9

4. Pengujian Transaksi dan Integrasi Payment Gateway

[Gambar 4.2.1 6 Halaman POS Mode Kasir]

[Gambar 4.2.1 8 Notifikasi Payment Berhasil]

Bagian paling krusial dalam pengujian fungsional ini adalah alur transaksi yang telah terintegrasi dengan metode pembayaran otomatis QRIS. Saat kasir selesai menginput detail cucian dan memilih metode pembayaran QRIS, sistem akan memicu fungsi GopayHandler yang memiliki tingkat validitas logika 100.0

5. Pengujian Dokumentasi Laporan (Export Excel)

[Gambar 4.2.1 9 Halaman Manajemen Laporan]

[Gambar 4.2.1 10 Laporan Riwayat Transaksi Harian]

[Gambar 4.2.1 11 Laporan Keuangan Harian]

Tahap akhir dari rangkaian pengujian fungsional adalah proses dokumentasi dan pembuatan laporan keuangan dalam format fisik. Admin melakukan filter terhadap periode laporan yang diinginkan, kemudian sistem menjalankan fungsi laporanExportHandler dengan cakupan pengujian 3.4

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Setelah melalui tahapan perancangan, implementasi, hingga pengujian terhadap Sistem Point of Sale (POS) WellClean Laundry, maka dapat ditarik beberapa kesimpulan utama:

1. Keberhasilan Integrasi Sistem: Aplikasi telah berhasil diimplementasikan dengan menggunakan bahasa pemrograman Go dan basis data Supabase, yang memungkinkan pengelolaan data operasional laundry dilakukan secara terpusat dan aman.
2. Validitas Fungsional: Seluruh fitur utama, mencakup manajemen data pelanggan, kategori layanan, pencatatan transaksi, hingga visualisasi laporan keuangan, telah dinyatakan berfungsi sesuai dengan kebutuhan pengguna.
3. Kualitas Logika Program: Berdasarkan hasil audit code coverage, fungsi-fungsi krusial yang menangani keamanan sistem (seperti enkripsi kata sandi) dan integrasi pembayaran otomatis (melalui webhook GoPay) telah mencapai tingkat validasi 100
4. Efisiensi Operasional: Sistem ini terbukti mampu mengotomatisasi perhitungan omzet harian maupun bulanan secara real-time, yang memberikan transparansi finansial bagi pemilik usaha.

5.2 Saran

Demi pengembangan sistem yang lebih optimal di masa mendatang, penulis menyarankan beberapa poin peningkatan sebagai berikut:

1. Akselerasi Cakupan Pengujian Unit: Melakukan optimasi pada modul-modul yang saat ini memiliki persentase coverage di bawah 50
2. Ekspansi Fitur Notifikasi Otomatis: Mengintegrasikan API pihak ketiga seperti WhatsApp Gateway atau layanan Email untuk memberikan pembaruan status cucian secara real-time kepada pelanggan, guna meningkatkan kualitas layanan dan transparansi operasional.
3. Implementasi Teknologi Progressive Web App (PWA): Mengadopsi mekanisme

PWA guna meningkatkan aksesibilitas aplikasi agar tetap mampu beroperasi secara stabil dan responsif, meskipun pengguna berada dalam kondisi konektivitas internet yang terbatas.

DAFTAR PUSTAKA

LAMPIRAN

Bibliometrik

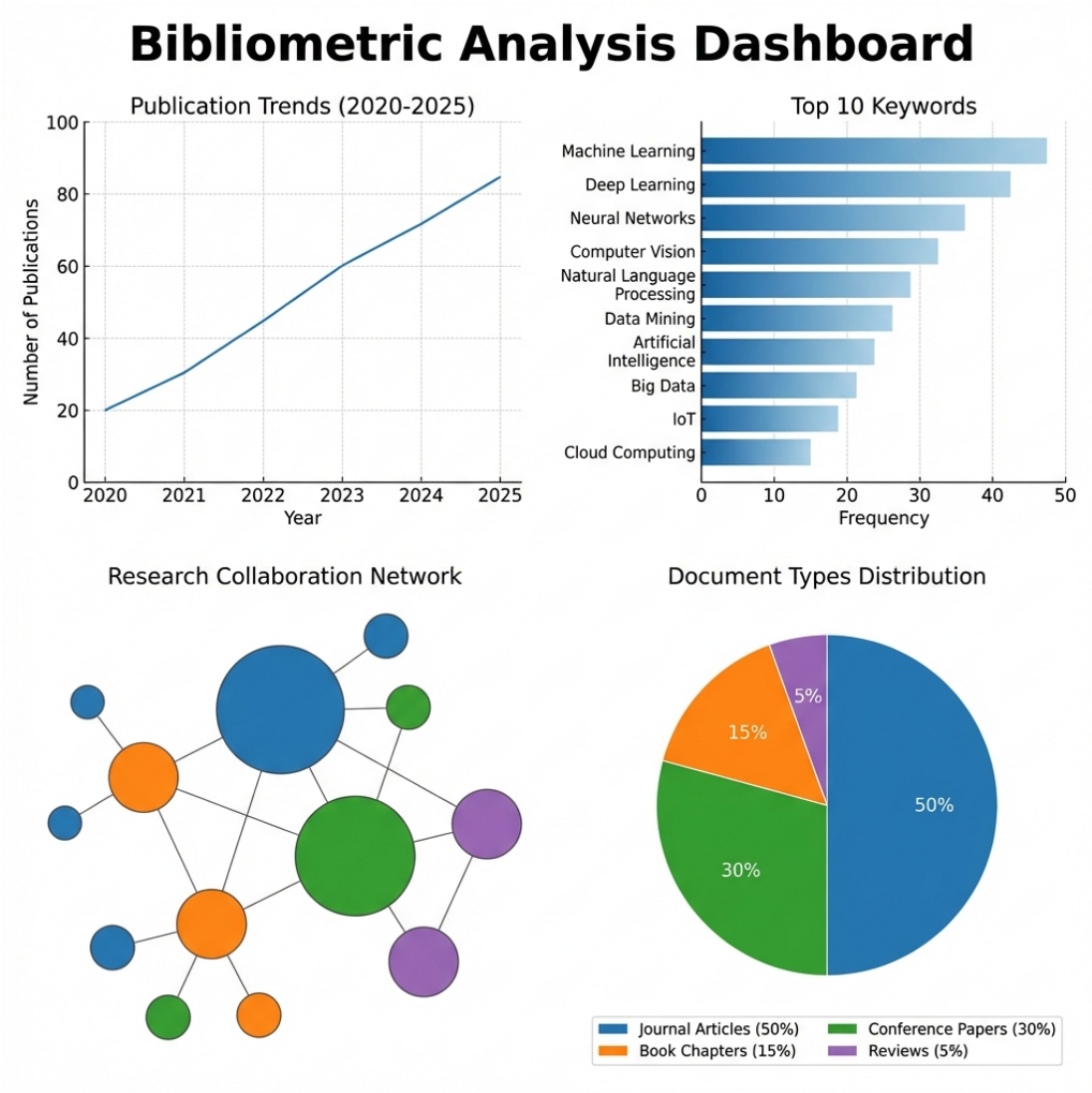


Figure 1: Visualisasi Overview Bibliometrik (Biblioshiny)

GLOSARIUM

GLOSARIUM TEKNIS

A

API (*Application Programming Interface*): Sekumpulan aturan dan protokol yang memungkinkan aplikasi *frontend* berkomunikasi dengan *server backend* untuk pertukaran data secara otomatis.

B

Backend: Bagian dari sistem yang menangani logika bisnis, pemrosesan data, dan komunikasi dengan *database* yang tidak terlihat langsung oleh pengguna.

C

Code Coverage: Metrik yang menunjukkan persentase baris kode program yang telah berhasil dieksekusi selama proses pengujian unit dijalankan.

D

Database (Supabase): Kumpulan data terorganisir yang disimpan secara digital dalam tabel (Pelanggan, Layanan, Transaksi) untuk mengelola operasional laundry.

G

Go (Golang): Bahasa pemrograman yang digunakan untuk membangun *backend* sistem karena efisiensi dan performanya dalam menangani *concurrency*.

J

JWT (*JSON Web Token*): Standar industri untuk pembuatan token akses yang digunakan dalam proses otentikasi keamanan saat admin atau kasir melakukan login.

U

Unit Testing: Metode pengujian di mana komponen terkecil dari kode diuji secara mandiri untuk memastikan fungsinya berjalan sesuai logika yang dirancang.

W

Webhook: Mekanisme yang digunakan oleh *payment gateway* untuk mengirimkan notifikasi data secara otomatis ke *server* saat transaksi pembayaran berhasil dideteksi.

GLOSARIUM NON-TEKNIS

A

Analisis Fungsional: Proses identifikasi kebutuhan utama sistem yang mencakup fitur-fitur inti seperti transaksi, manajemen pelanggan, dan pelaporan.

Analisis Non-Fungsional: Analisis terhadap aspek pendukung sistem agar beroperasi optimal, mencakup keamanan data, kinerja, dan kenyamanan pengguna (*usability*).

D

Dashboard: Halaman utama aplikasi yang menyajikan ringkasan statistik bisnis seperti total omset dan kinerja operasional secara visual.

F

Flowchart: Diagram yang merepresentasikan alur kerja atau algoritma proses bisnis laundry, mulai dari penerimaan order hingga penyerahan cucian.

P

Point of Sale (POS): Sistem digital yang digunakan untuk mengelola transaksi penjualan, pencatatan pesanan, dan pengelolaan stok secara terpadu.

Q

QRIS (*Quick Response Code Indonesian Standard*): Standar kode QR nasional untuk pembayaran digital yang memudahkan pelanggan melakukan transaksi melalui dompet digital.

R

Real-time: Kondisi pemrosesan data yang terjadi seketika, seperti pembaruan status pembayaran yang langsung terdeteksi oleh sistem tanpa perlu pengecekan manual.

S

Struk Transaksi: Bukti pembayaran digital atau fisik yang berisi rincian layanan laundry, berat cucian, dan total biaya yang harus dibayar pelanggan.